

Learn Low-Level Design in 25 Days: Day 14— Designing a Rate Limiter (Token Bucket, Leaky Bucket)

Ever wondered how APIs like Twitter or Stripe prevent abuse while staying blazing fast?

TechPreneur

Published October 26, 2025

Free: No

Read on Freedium · original

Contents

The Core Problem: "Too Much, Too Fast" 0

Two Common Strategies: Leaky Bucket vs. Token Bucket 0

1. Leaky Bucket Algorithm 0

2. Token Bucket Algorithm 0

Visualizing the Difference 0

Mini Case Study: How GitHub Enforces API Limits 0

Try It Yourself 0

Key Takeaways 0

What's Next 0

Imagine you've just built a killer API. Developers love it. Users start integrating it everywhere. But then… someone writes a script that makes **10,000 requests per second**, and suddenly, your servers start gasping for air.

Your system crashes. Other users are throttled. Support tickets explode.

This is where **Rate Limiting** steps in — the unsung hero that keeps APIs reliable, fair, and scalable.

Welcome to **Day 14 of "Learn Low-Level Design in 25 Days."** Today, we'll dive into **Rate Limiter Design**, exploring two powerful algorithms that form its backbone — the **Token Bucket** and the **Leaky Bucket**.



By the end, you'll understand how to design and implement a rate limiter that keeps your system stable under pressure — just like how Netflix controls streaming requests or how GitHub enforces API limits.

The Core Problem: "Too Much, Too Fast"

Let's start simple.

When a user sends requests to your service, you want to ensure fairness and prevent overload. But how do you decide how many requests are "too many"?

Without control, even legitimate clients can accidentally flood your servers — think of an auto-refreshing dashboard or a background script gone rogue.

So, we need a **Rate Limiter** — a mechanism that allows requests **up to a defined rate**, then **drops, delays, or throttles** excess ones.

Think of it like a **security guard at a nightclub** — only a certain number of people can enter per minute, no matter how long the line gets.

Two Common Strategies: Leaky Bucket vs. Token Bucket

Let's look at the two classic models used across distributed systems.

1. Leaky Bucket Algorithm

Analogy: Imagine a bucket with a small hole at the bottom. Water (requests) flows in from the top. The bucket leaks water at a steady rate. If you pour too much water too quickly, the bucket overflows — and new requests are dropped.

Behavior:

- Processes requests at a **fixed rate**.
- Smooths out traffic bursts.
- Discards excess requests when capacity is full.

Implementation Sketch:

```
class LeakyBucketRateLimiter {
    private final int capacity;
    private final int leakRate; // requests per second
    private int water; // current load
    private long lastLeakTimestamp;

    public LeakyBucketRateLimiter(int capacity, int leakRate) {
        this.capacity = capacity;
        this.leakRate = leakRate;
        this.water = 0;
        this.lastLeakTimestamp = System.currentTimeMillis();
    }

    public synchronized boolean allowRequest() {
        leak();
        if (water < capacity) {
            water++;
            return true;
        }
        return false;
    }

    private void leak() {
        long now = System.currentTimeMillis();
        long elapsed = now - lastLeakTimestamp;
        int leaked = (int) (elapsed * leakRate / 1000);
        water = Math.max(0, water - leaked);
        if (leaked > 0) lastLeakTimestamp = now;
    }
}
```

Use case: Perfect for **smoothing out bursty traffic** — like handling logs or queue workers.

2. Token Bucket Algorithm

Analogy: Imagine a bucket that collects tokens at a steady rate (say 5 tokens per second). Each request consumes one token. If the bucket is empty, requests must wait or be denied.

Unlike Leaky Bucket, this allows **bursts** — as long as tokens are available.

Behavior:

- Allows bursts up to bucket capacity.
- Replenishes tokens at a steady rate.
- More flexible than Leaky Bucket.

Implementation Sketch:

```
class TokenBucketRateLimiter {
    private final int capacity;
    private final int refillRate; // tokens per second
    private double tokens;
    private long lastRefillTimestamp;

    public TokenBucketRateLimiter(int capacity, int refillRate) {
        this.capacity = capacity;
        this.refillRate = refillRate;
        this.tokens = capacity;
        this.lastRefillTimestamp = System.currentTimeMillis();
    }

    public synchronized boolean allowRequest() {
        refill();
        if (tokens >= 1) {
            tokens -= 1;
            return true;
        }
        return false;
    }

    private void refill() {
        long now = System.currentTimeMillis();
        double elapsed = (now - lastRefillTimestamp) / 1000.0;
        double added = elapsed * refillRate;
        tokens = Math.min(capacity, tokens + added);
        if (added > 0) lastRefillTimestamp = now;
    }
}
```

Use case: Ideal for **APIs or user-facing systems** where small bursts are okay — e.g., a user quickly clicking multiple times or fetching a short sequence of resources.

Visualizing the Difference

```
Leaky Bucket:
Input --> [ Bucket ] --> Leak steadily --> Output (constant rate)

Token Bucket:
Tokens added steadily --> [ Bucket ]
Requests consume tokens --> Output (bursty, controlled)
```

- **Leaky Bucket:** smooths output rate.
- **Token Bucket:** controls burstiness.

Mini Case Study: How GitHub Enforces API Limits

GitHub enforces API limits of 5000 requests per hour per user. Behind the scenes, this likely works with a **Token Bucket**, where:

- Each API call costs one token.
- Tokens are replenished every second.
- If your bucket is empty, you wait or get a `429 Too Many Requests`.

This approach keeps the API reliable while letting users make quick bursts when needed (like fetching 50 repos at once).

Try It Yourself

Implement a rate limiter for your REST API that allows:

- 10 requests per second (token bucket style)
- Burst of up to 20 requests.

Bonus: Add a global limiter shared across multiple users (simulate using a static variable or Redis).

Reflection: Which model fits better for your system — consistent throughput or burst-friendly flexibility?

Key Takeaways

- **Rate Limiting** protects systems from overload and ensures fair usage.
- **Leaky Bucket** smooths traffic — fixed rate, steady flow.
- **Token Bucket** allows bursts — dynamic, flexible.
- Always balance **user experience** (speed) with **system stability** (control).
- Distributed systems often implement rate limiters using **Redis**, **sliding windows**, or **cluster-aware counters**.

What's Next

Tomorrow, we'll move into something every scalable system needs — **Designing a Notification Service (Email, SMS, Push)** and see how to structure it for reliability and extensibility.

Follow the full series here: Low-Level Design in 25 Days: A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8) Save the list to continue your 25-day journey toward mastering real-world LLD.